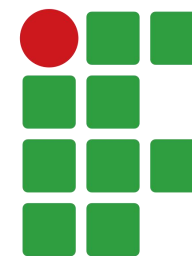


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

ListView

- Exibe uma coleção de Views em uma lista com rolagem vertical, posicionando cada item imediatamente abaixo do item anterior.
- Para utilizar um ListView, basta adicionar uma View do tipo ListView ao arquivo XML de layout:

```
<ListView  
    android:id="@+id/list_view"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```

ListView

New Project

×

Empty Activity

Create a new empty activity with Jetpack Compose

Name

ListAndRecyclerView

Package name

leite.sampaio.lucas.listandrecyclerview

Save location

/home/lucas/AndroidStudioProjects/ListAndRecyclerView2

📁

Minimum SDK

API 24 ("Nougat"; Android 7.0)

⌵

ⓘ

Your app will run on approximately 99,2% of devices.

[Help me choose](#)

Build configuration language ⓘ

Kotlin DSL (build.gradle.kts) [Recommended]

⌵

Previous

Next

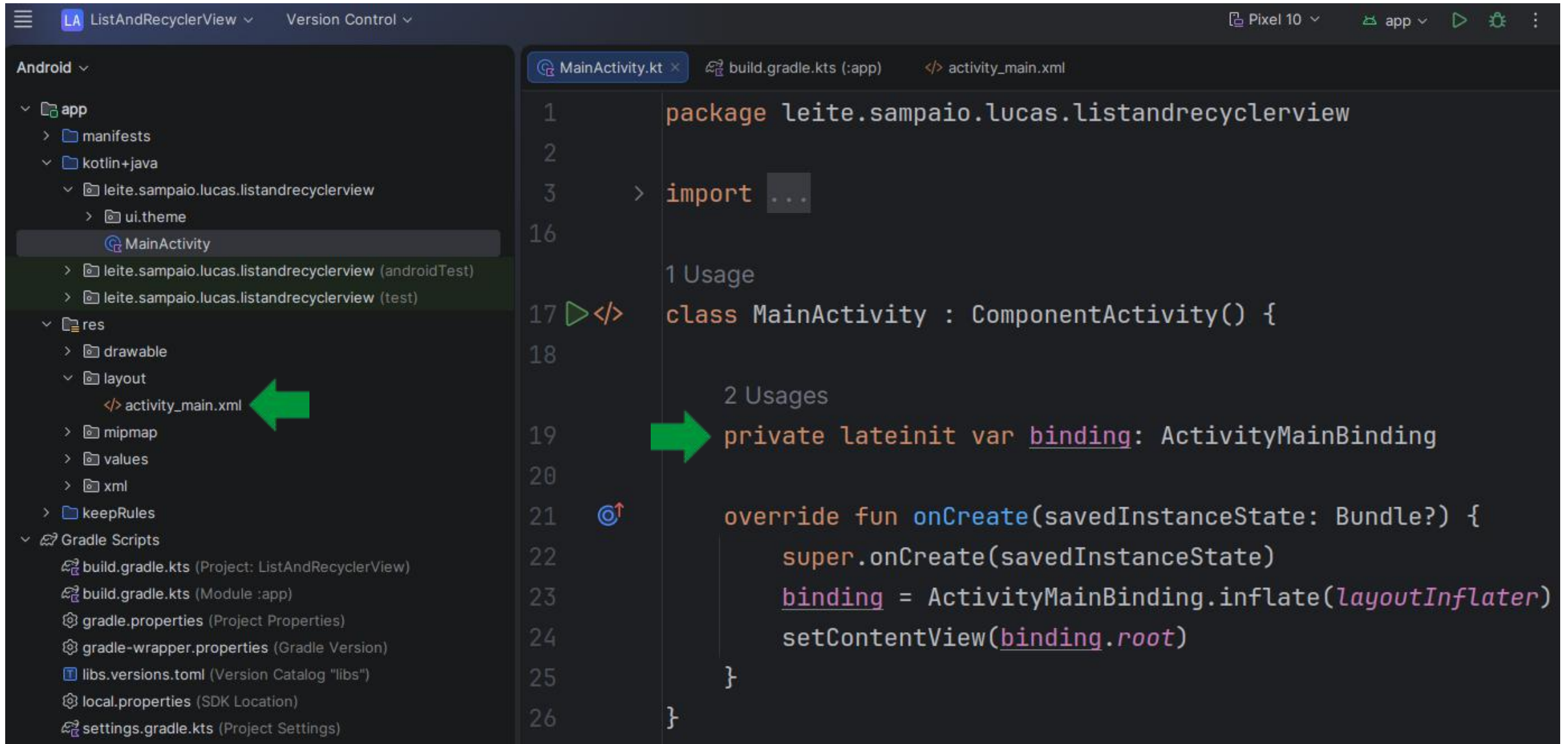
Cancel

Finish



ListView

```
viewBinding{  
    enable = true  
}
```



Android ▾

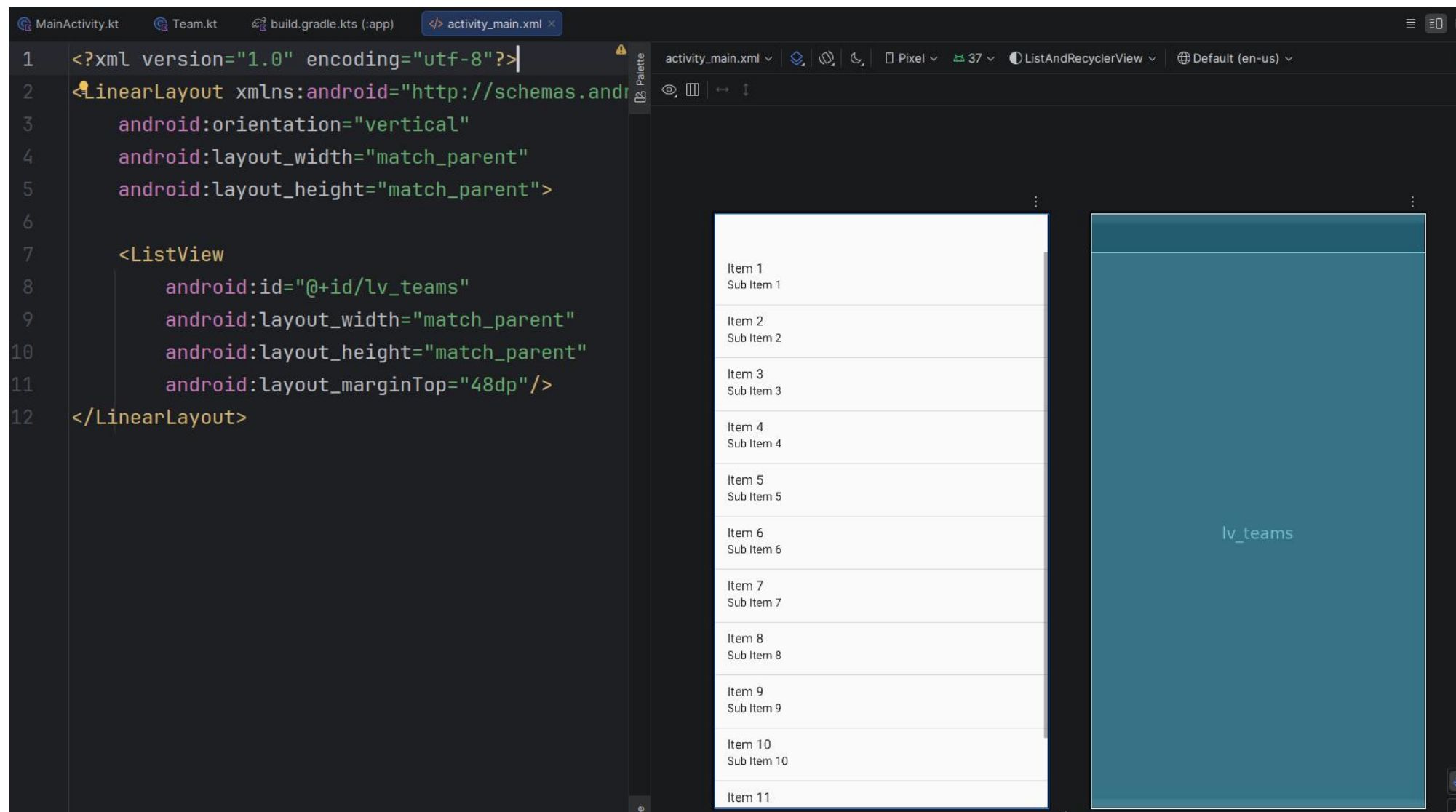
- app
 - manifests
 - kotlin+java
 - leite.sampaio.lucas.listandrecyclerview
 - ui.theme
 - MainActivity
 - leite.sampaio.lucas.listandrecyclerview (androidTest)
 - leite.sampaio.lucas.listandrecyclerview (test)
 - res
 - drawable
 - layout
 - activity_main.xml ←
 - mipmap
 - values
 - xml
 - keepRules
- Gradle Scripts
 - build.gradle.kts (Project: ListAndRecyclerView)
 - build.gradle.kts (Module :app)
 - gradle.properties (Project Properties)
 - gradle-wrapper.properties (Gradle Version)
 - libs.versions.toml (Version Catalog "libs")
 - local.properties (SDK Location)
 - settings.gradle.kts (Project Settings)

Pixel 10 ▾ app ▾ ▶ ⚙ ⋮

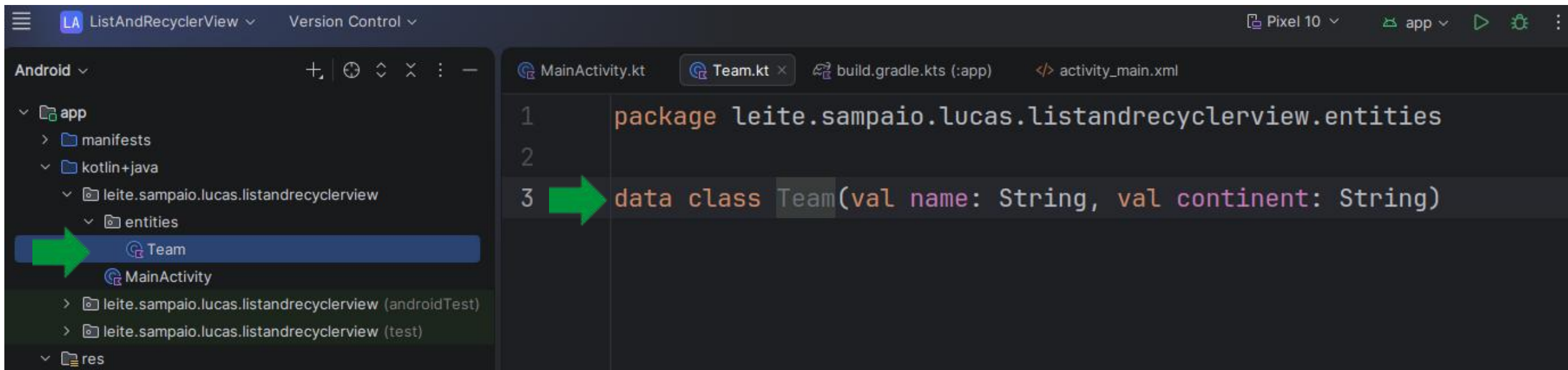
MainActivity.kt × build.gradle.kts (:app) </> activity_main.xml

```
1 package leite.sampaio.lucas.listandrecyclerview  
2  
3 > import ...  
16  
17 1 Usage  
18 class MainActivity : AppCompatActivity() {  
19 2 Usages  
20     private lateinit var binding: ActivityMainBinding  
21  
22     override fun onCreate(savedInstanceState: Bundle?) {  
23         super.onCreate(savedInstanceState)  
24         binding = ActivityMainBinding.inflate(layoutInflater)  
25         setContentView(binding.root)  
26     }  
}
```

ListView



Modelando o objeto Team



Data Classes

- Data classes são indicadas para classes cujo principal objetivo é armazenar dados.
- O Kotlin gera automaticamente várias funcionalidades úteis:
 - `equals()`: Compara dois objetos para verificar se seus dados são iguais.
 - `hashCode()`: Gera um código hash baseado nas propriedades dos dados.
 - `toString()`: Gera uma representação em string dos dados da classe.
 - `copy()`: Permite criar cópias do objeto com algumas propriedades modificadas.

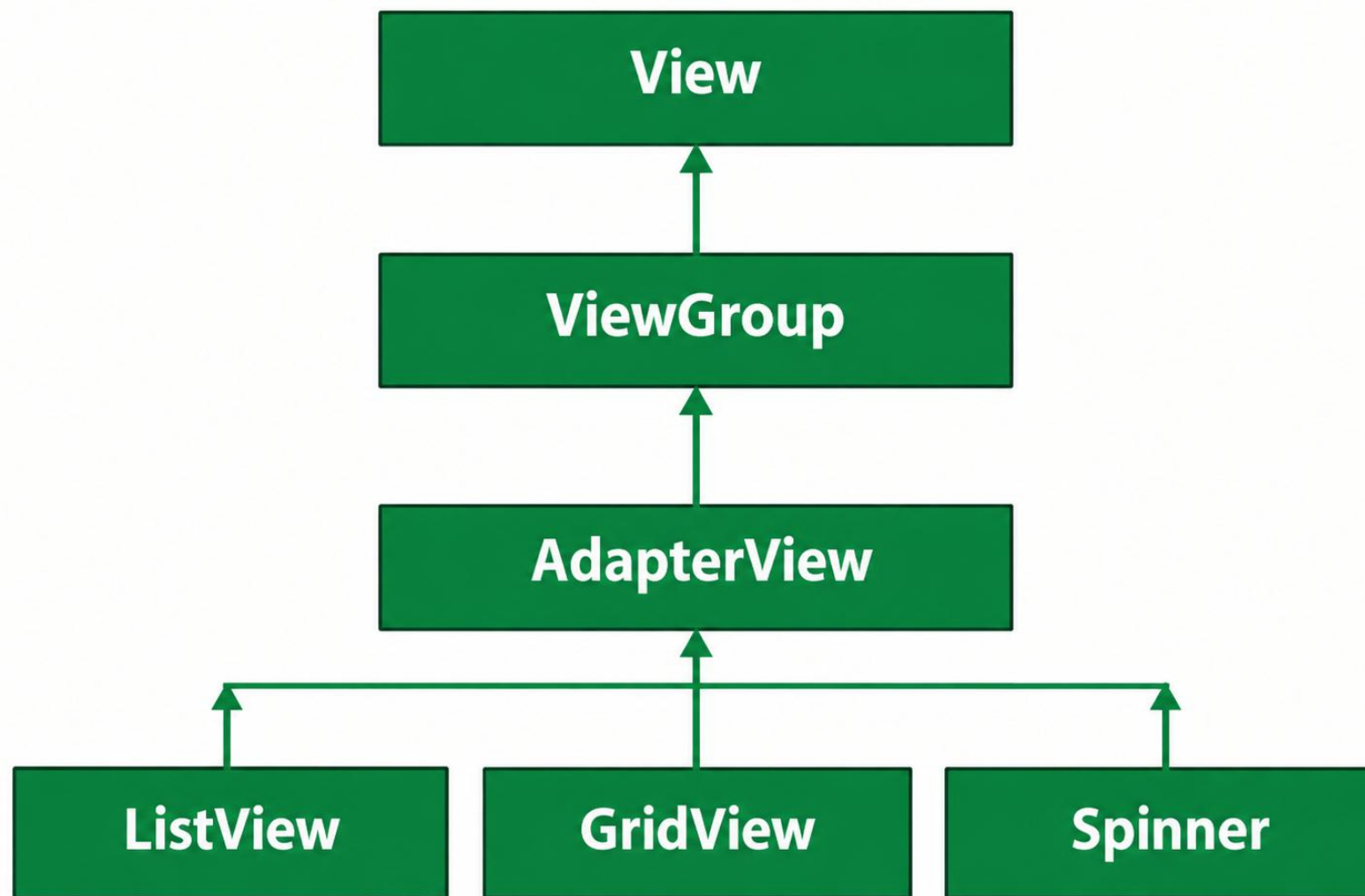
Obtendo a referência da ListView na Activity

```
class MainActivity : ComponentActivity() {  
  
    3 Usages  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        ➡ val listTeams = binding.lvTeams  
    }  
}
```

Adapters

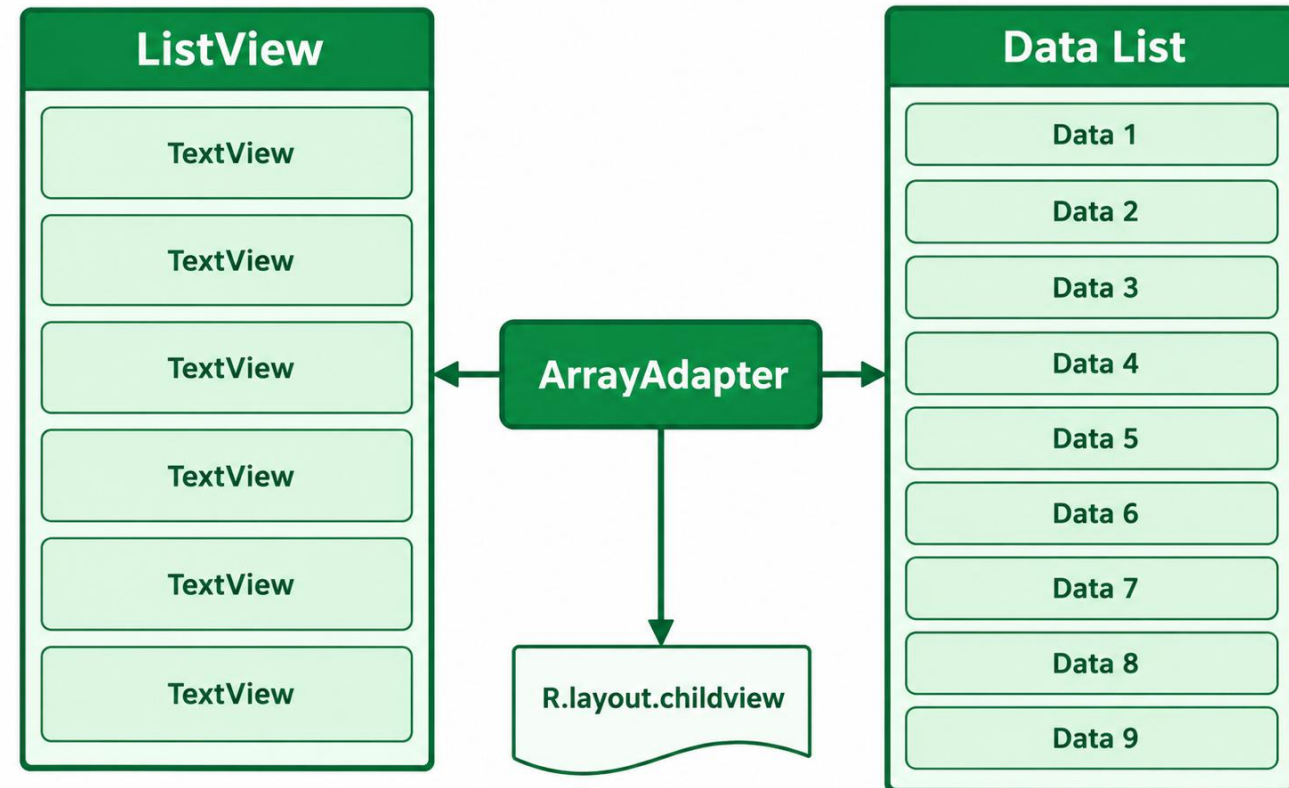
- No Android, um Adapter atua como uma ponte entre uma fonte de dados e uma AdapterView, fornecendo os dados necessários para sua exibição.
 - AdapterView é uma classe-base para componentes que exibem coleções de dados utilizando um Adapter, como ListView, GridView e Spinner.
- O Adapter também é responsável por fornecer ou criar uma View para representar cada item da coleção de dados.

Adapters



Data Classes

- ArrayAdapter é um adapter simples fornecido pelo Android para exibir uma coleção de dados em componentes como ListView.
- Por padrão, utiliza o método toString() de cada objeto para definir o texto exibido no TextView de cada item.



Definindo o adaptador da lista

```
class MainActivity : ComponentActivity() {  
  
    3 Usages  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        val listTeams = binding.lvTeams  
        val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, teams)  
    }  
}
```

Para criar um ArrayAdapter, é necessário informar um Context, geralmente a Activity atual, o recurso de layout que será utilizado para representar cada item da lista e a coleção de dados que será exibida.

Criando os objetos Team

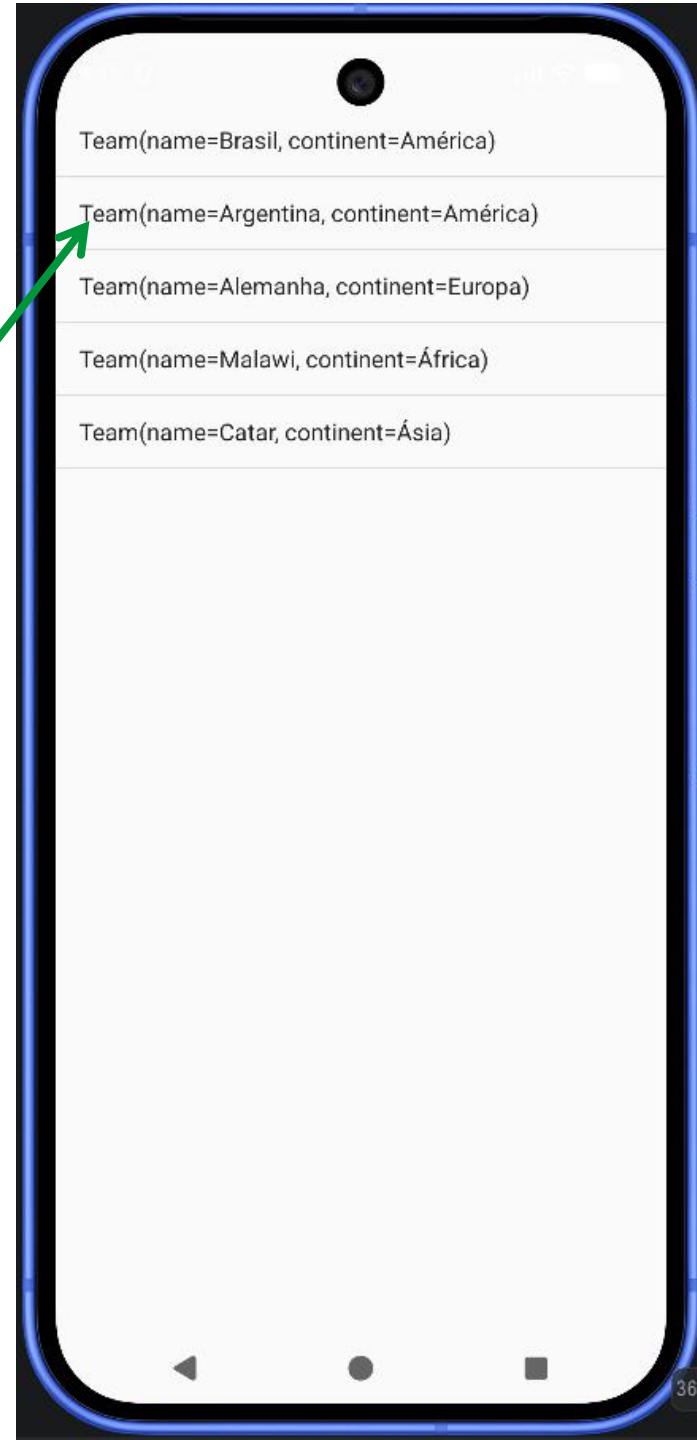
```
class MainActivity : AppCompatActivity() {  
  
    3 Usages  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        ➡ val teams = arrayOf(  
            Team(name = "Brasil", continent = "América"),  
            Team(name = "Argentina", continent = "América"),  
            Team(name = "Alemanha", continent = "Europa"),  
            Team(name = "Malawi", continent = "África"),  
            Team(name = "Catar", continent = "Ásia")  
        )  
  
        val listTeams = binding.lvTeams  
        val adapter = ArrayAdapter(context = this, resource = android.R.layout.simple_list_item_1, objects = teams)  
    }  
}
```

Vinculando o adapter a ListView

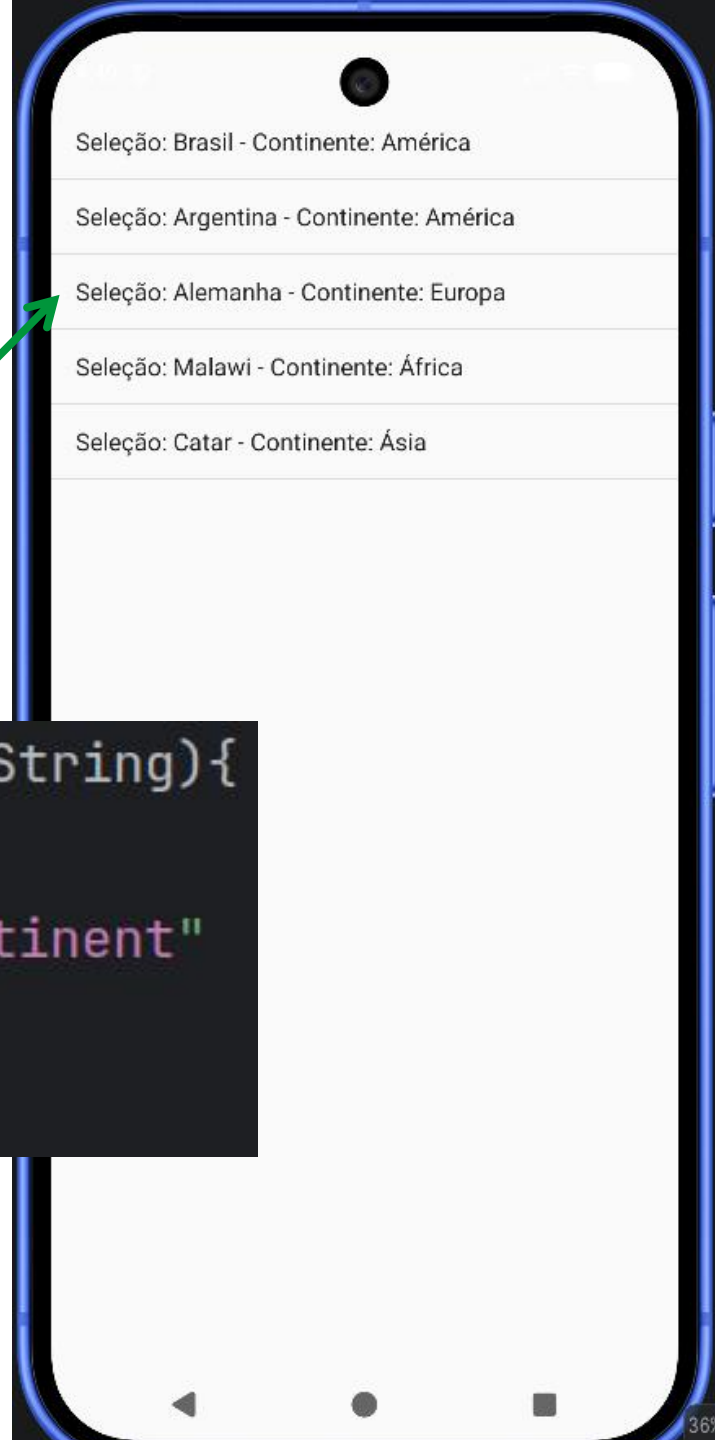
```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    val teams = arrayOf(  
        Team(name = "Brasil", continent = "América"),  
        Team(name = "Argentina", continent = "América"),  
        Team(name = "Alemanha", continent = "Europa"),  
        Team(name = "Malawi", continent = "África"),  
        Team(name = "Catar", continent = "Ásia")  
    )  
  
    val listTeams = binding.lvTeams  
    val adapter = ArrayAdapter(context = this, resource = android.R.layout.simple_list_item_1, objects = teams)  
    listTeams.adapter = adapter  
}
```

Executando o app

Por que os dados aparecem nesse formato?



Sobrescrevendo o método toString()



```
data class Team(val name: String, val continent: String){  
    ➔ override fun toString(): String {  
        return "Seleção: $name - Continente: $continent"  
    }  
}
```

ListView x RecyclerView

- ListView: componente mais antigo e simples para exibir listas, porém possui menos flexibilidade e eficiência para listas grandes ou layouts mais complexos.
 - <https://developer.android.com/reference/android/widget/ListView>
- RecyclerView: componente mais moderno e recomendado para listas, pois reutiliza as views dos itens, oferece maior flexibilidade de layout e facilita recursos como diferentes tipos de itens, animações e atualização dos dados.
 - <https://developer.android.com/develop/ui/views/layout/recyclerview>
- Adapter: no ListView, é comum utilizar ArrayAdapter para listas simples; no RecyclerView, é necessário criar um Adapter e um ViewHolder próprios, proporcionando maior controle sobre a representação de cada item.

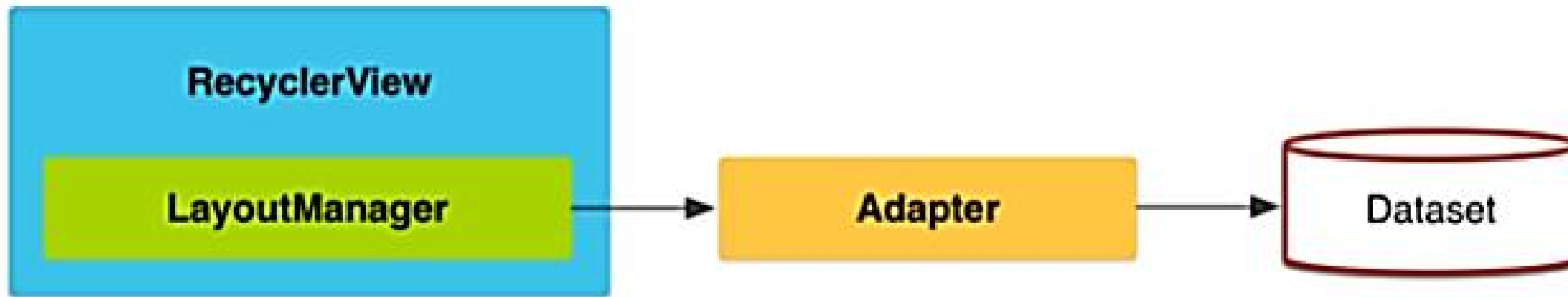
RecyclerView

- RecyclerView é um componente utilizado para exibir coleções de dados de forma eficiente, por meio de um Adapter.
- É uma alternativa mais flexível ao ListView e ao GridView, oferecendo uma estrutura mais extensível.
- Um de seus principais diferenciais é a possibilidade de utilizar diferentes gerenciadores de layout (LayoutManager), permitindo organizar os itens na horizontal, na vertical ou em uma grade.
- É especialmente adequado para listas cujos itens podem ser atualizados, adicionados ou removidos durante a execução do aplicativo, em resposta a ações do usuário ou a eventos externos.

RecyclerView

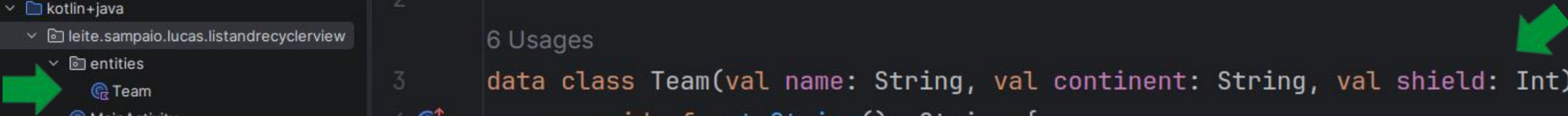
- O RecyclerView, como o próprio nome indica, reutiliza as Views dos itens à medida que elas deixam de ser necessárias na tela.
- Quando um item sai da área visível, sua View não precisa ser destruída.
- Essa View pode ser reutilizada para representar outro item que entra na área visível, atualizando apenas os dados exibidos.
- Essa estratégia reduz a quantidade de Views que precisam ser criadas e melhora o desempenho e a capacidade de resposta do aplicativo.

RecyclerView



O RecyclerView não define como os itens da lista devem ser organizados na tela. Essa responsabilidade é delegada a um LayoutManager, que determina a disposição e o posicionamento dos itens.

Modificando a definição do objeto Team



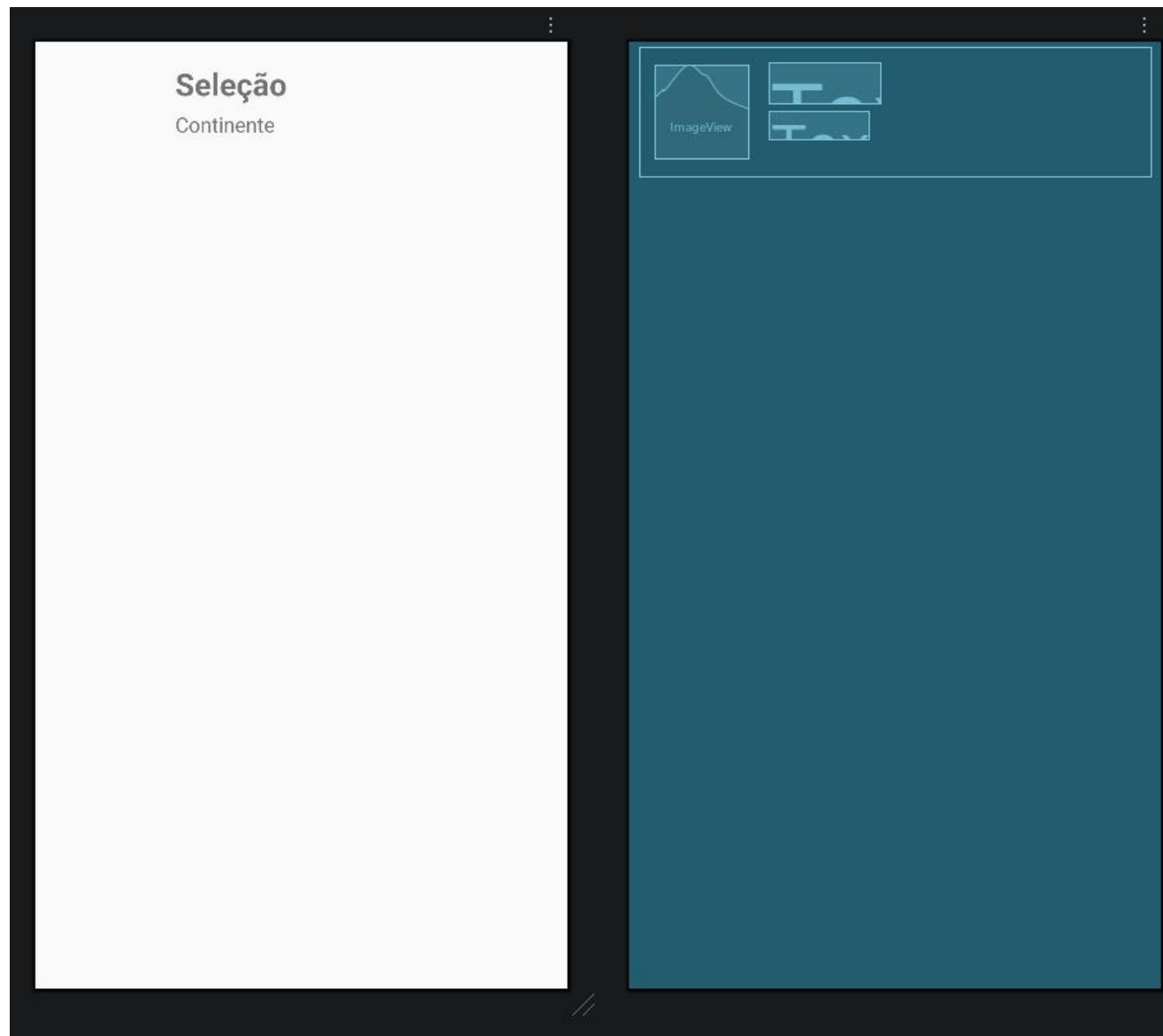
Android ▾

- app
 - manifests
 - kotlin+java
 - leite.sampaio.lucas.listandrecyclerview
 - entities
 - Team**
 - MainActivity
 - leite.sampaio.lucas.listandrecyclerview (a)
 - leite.sampaio.lucas.listandrecyclerview (te)
 - java (generated)
 - res
 - drawable
 - layout

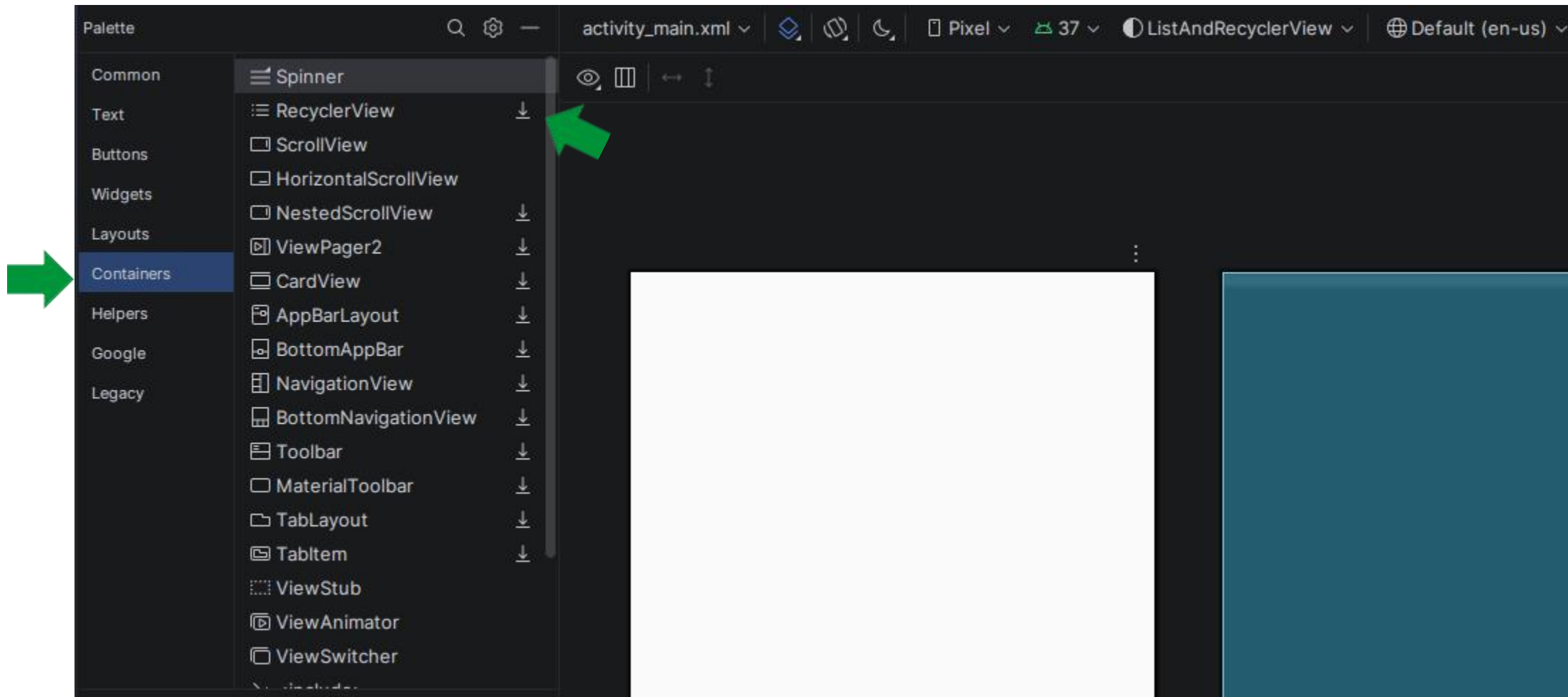
MainActivity.kt Team.kt × build.gradle.kts (:app) activity_main.xml

```
1 package leite.sampaio.lucas.listandrecyclerview.entities
2
3 6 Usages
4 data class Team(val name: String, val continent: String, val shield: Int){
5     override fun toString(): String {
6         return "Seleção: $name - Continente: $continent"
7     }
8 }
```

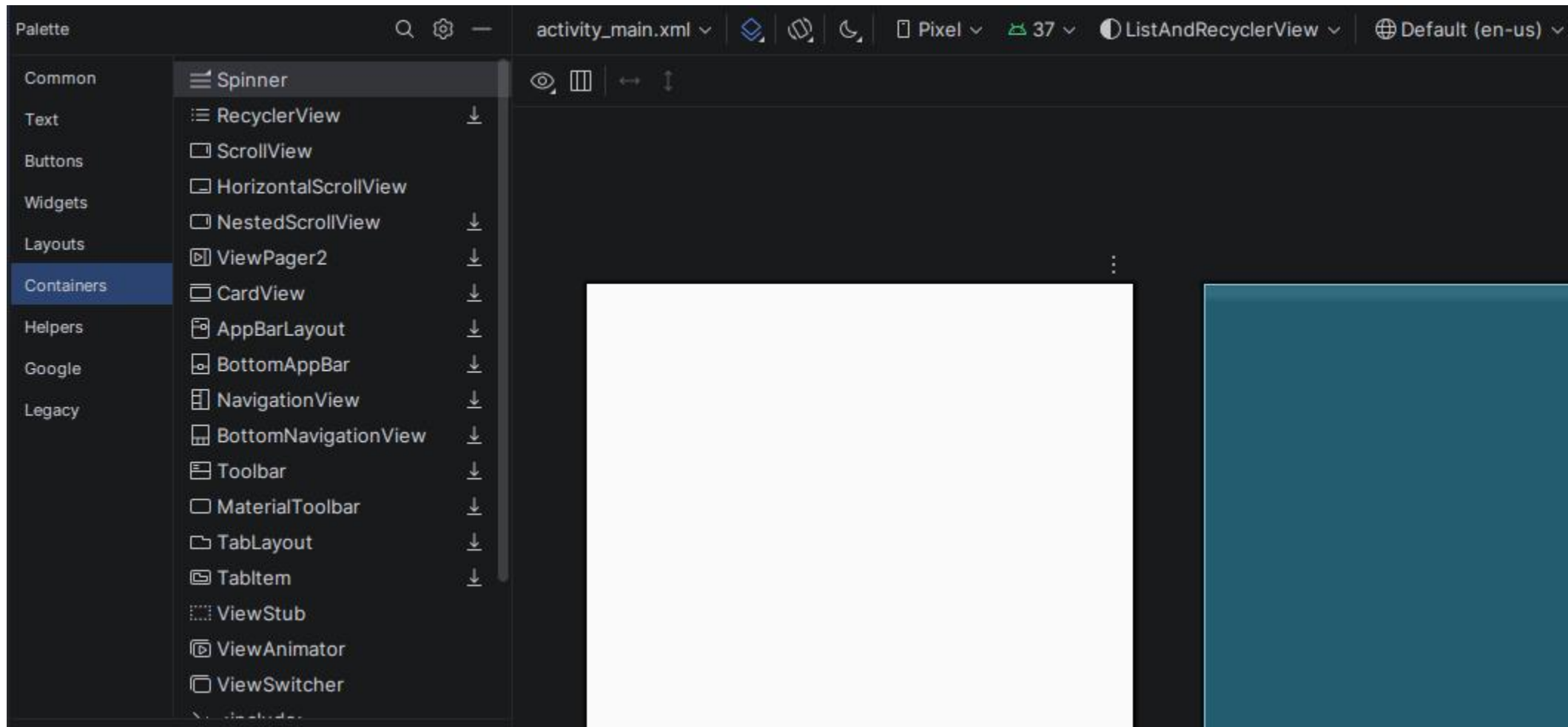
Definindo o layout do item



Adicionando o RecyclerView ao Layout da Activity

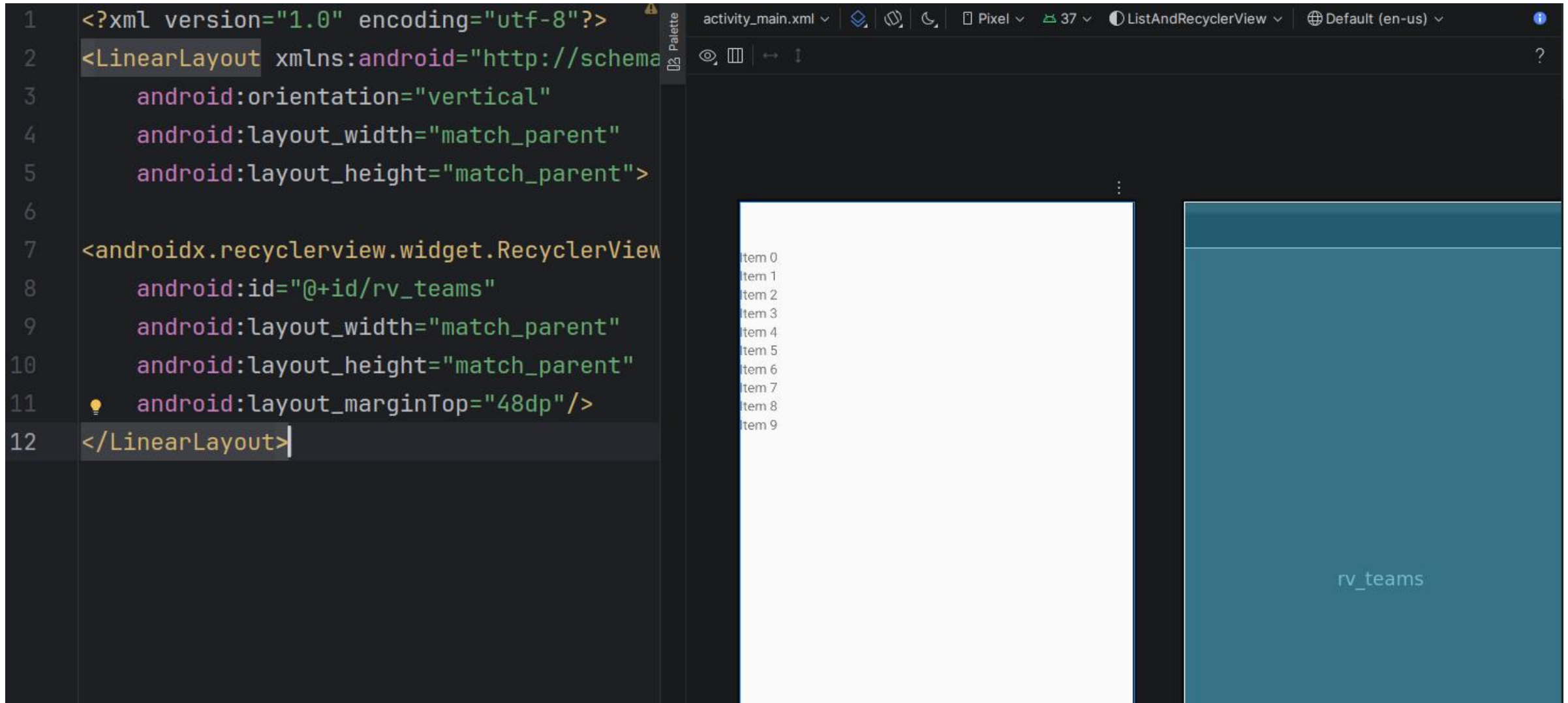


Adicionando o RecyclerView ao Layout da Activity



```
implementation(libs.androidx.recyclerview)
```

Adicionando o RecyclerView ao Layout da Activity



The screenshot displays the Android Studio environment. On the left, the XML editor shows the following code for `activity_main.xml`:

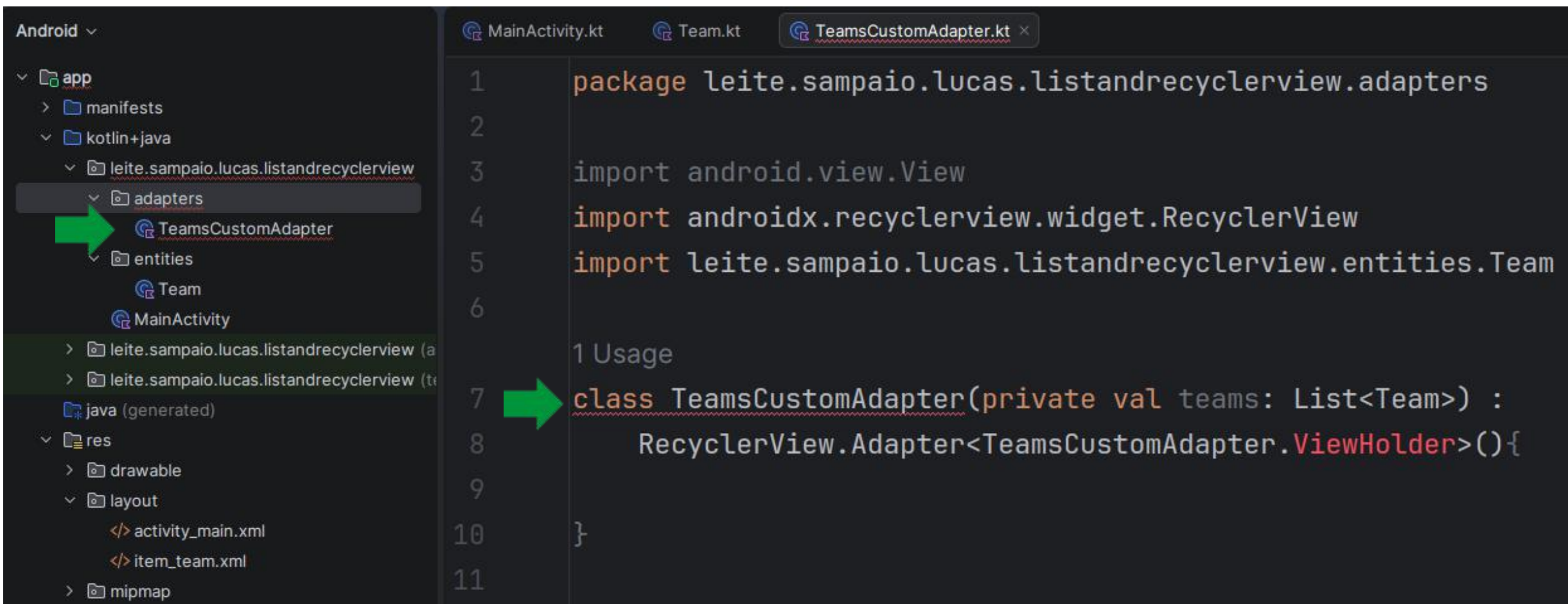
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:orientation="vertical"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7     <androidx.recyclerview.widget.RecyclerView
8         android:id="@+id/rv_teams"
9         android:layout_width="match_parent"
10        android:layout_height="match_parent"
11        android:layout_marginTop="48dp"/>
12 </LinearLayout>
```

On the right, the visual layout preview shows a vertical list of 10 items, labeled "Item 0" through "Item 9". The RecyclerView component is highlighted in blue and labeled "rv_teams". The interface also shows the "Palette" tab on the left and the "activity_main.xml" file selected in the top bar.

RecyclerView

- O RecyclerView utiliza o padrão ViewHolder para armazenar referências às Views que compõem cada item da lista.
- O ViewHolder evita que seja necessário localizar novamente essas Views com `findViewById()` sempre que os dados de um item forem atualizados.
- Dessa forma, as Views podem ser reutilizadas para exibir diferentes itens à medida que o usuário percorre a lista, contribuindo para um melhor desempenho.

Criando um Adapter personalizado



The screenshot displays the Android Studio IDE with the following components:

- Left Sidebar (Project Structure):** Shows the project hierarchy. The 'adapters' folder is expanded, and 'TeamCustomAdapter.kt' is highlighted with a green arrow.
- Top Editor Tabs:** Shows 'MainActivity.kt', 'Team.kt', and 'TeamsCustomAdapter.kt'.
- Main Editor (TeamsCustomAdapter.kt):** Contains the following Kotlin code:

```
1 package leite.sampaio.lucas.listandrecyclerview.adapters
2
3 import android.view.View
4 import androidx.recyclerview.widget.RecyclerView
5 import leite.sampaio.lucas.listandrecyclerview.entities.Team
6
7 1 Usage
8 class TeamsCustomAdapter(private val teams: List<Team>) :
9     RecyclerView.Adapter<TeamsCustomAdapter.ViewHolder>(){
10 }
11
```

Criando a classe ViewHolder para armazenar as referências

```
class TeamsCustomAdapter(private val teams: List<Team>) :  
    RecyclerView.Adapter<TeamsCustomAdapter.ViewHolder>(){
```

1 Usage

```
➔ class ViewHolder(view: View) : RecyclerView.ViewHolder(itemView = view){  
    val tvName: TextView = view.findViewById(R.id.tv_name)  
    val tvContinent: TextView = view.findViewById(R.id.tv_continent)  
    val ivShield: ImageView = view.findViewById(R.id.iv_teamShield)  
}  
}
```

Implementando os métodos de RecyclerView.Adapter

- Para criar o adapter, é necessário estender RecyclerView.Adapter<> e implementar três métodos principais:
 - onCreateViewHolder(): é chamado quando o RecyclerView precisa criar um novo ViewHolder. Nesse método, a View que representa o item é criada e o ViewHolder é inicializado.
 - onBindViewHolder(): é chamado para associar os dados de um item a um ViewHolder. Nesse método, os dados são obtidos a partir da posição do item e utilizados para atualizar as Views.
 - getItemCount(): retorna a quantidade de itens disponíveis no conjunto de dados, permitindo que o RecyclerView determine quantos itens precisam ser exibidos.

Implementando os métodos de RecyclerView.Adapter

```
class TeamsCustomAdapter(private val teams: List<Team>) :  
    RecyclerView.Adapter<TeamsCustomAdapter.ViewHolder>(){  
  
    ➡ override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {  
        TODO(reason = "Not yet implemented")  
    }  
  
    ➡ override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
        TODO(reason = "Not yet implemented")  
    }  
  
    ➡ override fun getItemCount(): Int {  
        TODO(reason = "Not yet implemented")  
    }  
  
    3 Usages  
    class ViewHolder(view: View) : RecyclerView.ViewHolder(itemView = view){  
        val tvName: TextView = view.findViewById(R.id.tv_name)  
        val tvContinent: TextView = view.findViewById(R.id.tv_continent)  
        val ivShield: ImageView = view.findViewById(R.id.iv_teamShield)  
    }  
}
```

Implementando os métodos de RecyclerView.Adapter

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {  
    val view = LayoutInflater.from(parent.context)  
        .inflate(resource = R.layout.item_team, root = parent, attachToRoot = false)  
    return ViewHolder(view)  
}
```

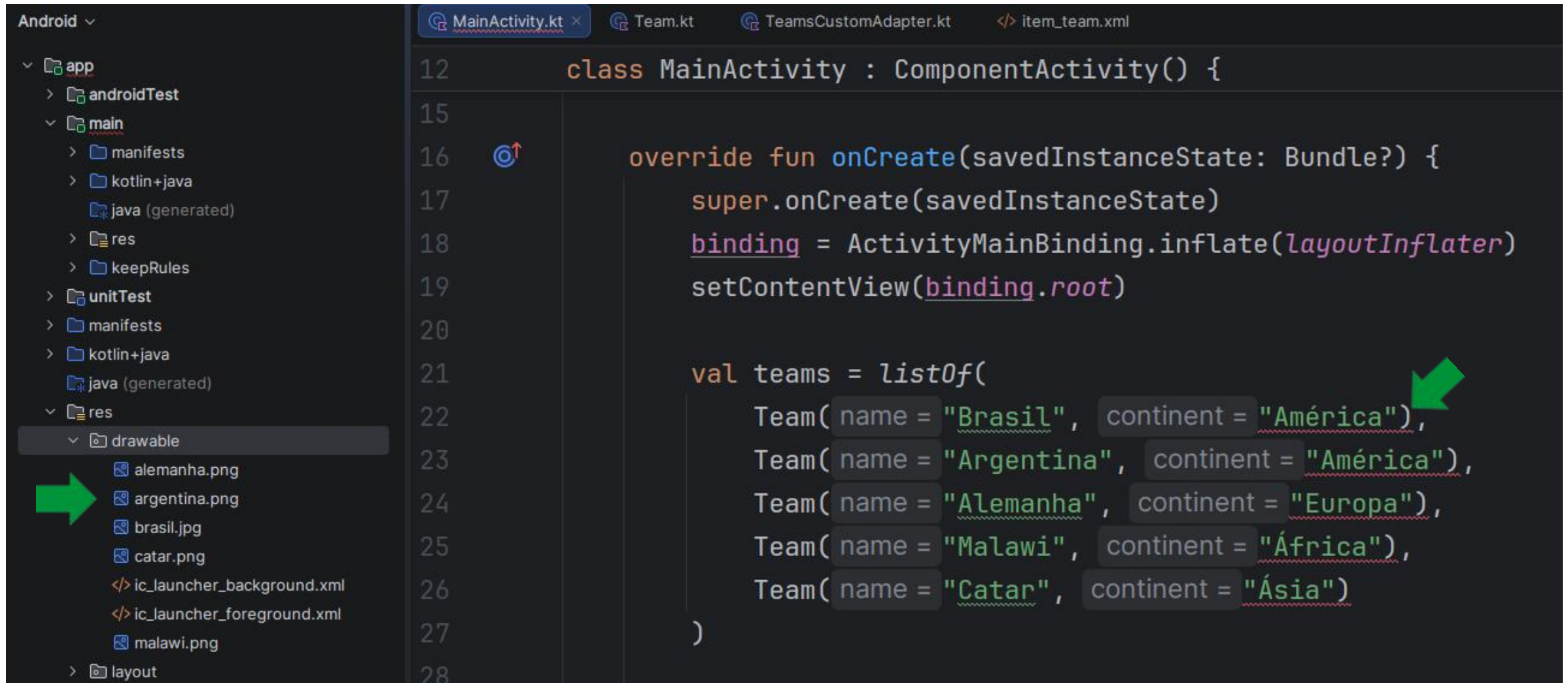
Implementando os métodos de RecyclerView.Adapter

```
override fun getItemCount(): Int {  
    return teams.size  
}
```

Implementando os métodos de RecyclerView.Adapter

```
override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
    val team = teams[position]  
    holder.tvName.text = team.name  
    holder.tvContinent.text = team.continent  
    holder.ivShield.setImageResource(team.shield)  
}
```

Modificando a lista de Times



```
12 class MainActivity : ComponentActivity() {
15
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18         binding = ActivityMainBinding.inflate(layoutInflater)
19         setContentView(binding.root)
20
21         val teams = listOf(
22             Team(name = "Brasil", continent = "América"),
23             Team(name = "Argentina", continent = "América"),
24             Team(name = "Alemanha", continent = "Europa"),
25             Team(name = "Malawi", continent = "África"),
26             Team(name = "Catar", continent = "Ásia")
27         )
28     }
```

Fonte: <https://escudosfc.com.br/>

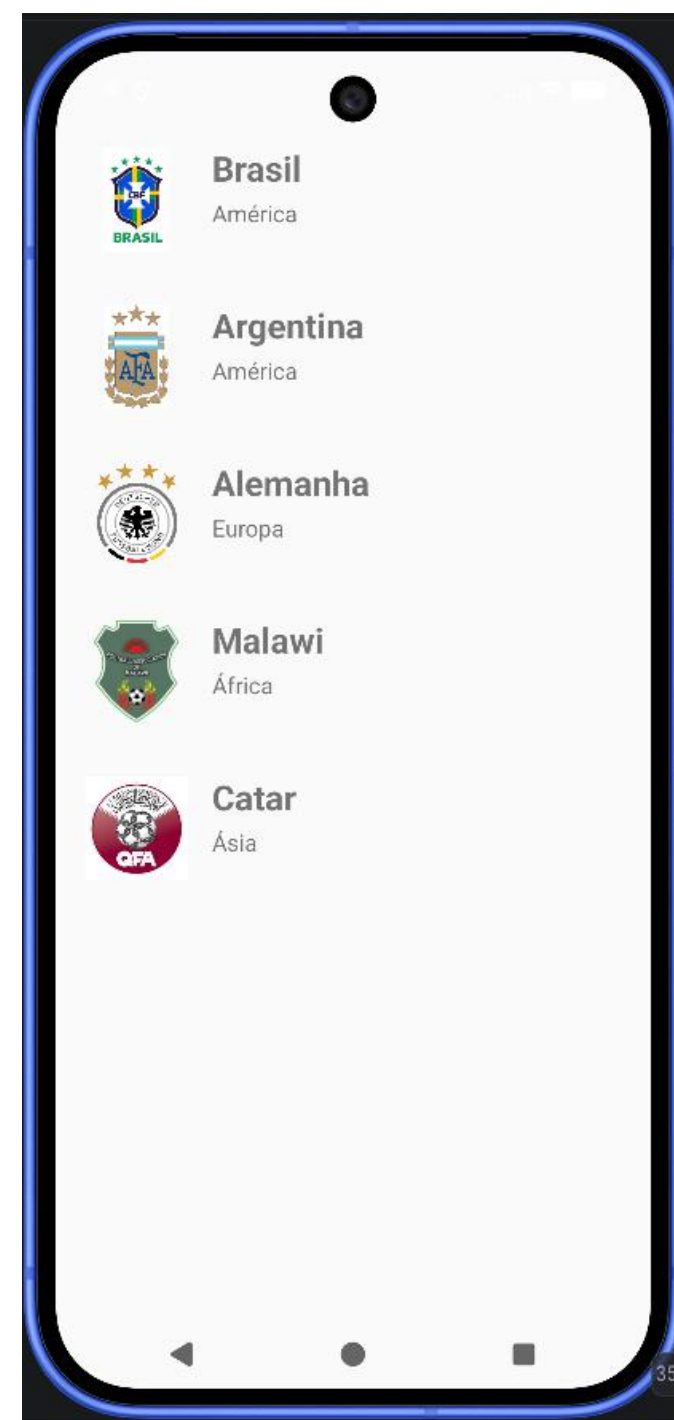
Modificando a lista de Times

```
21      val teams = listOf(  
22          Team(name = "Brasil", continent = "América", shield = R.drawable.brasil),  
23          Team(name = "Argentina", continent = "América", shield = R.drawable.argentina),  
24          Team(name = "Alemanha", continent = "Europa", shield = R.drawable.alemanha),  
25          Team(name = "Malawi", continent = "África", shield = R.drawable.malawi),  
26          Team(name = "Catar", continent = "Ásia", shield = R.drawable.catar)  
27      )
```

Definindo o Adapter e o LayoutManager

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    val teams = listOf(  
        Team(name = "Brasil", continent = "América", shield = R.drawable.brasil),  
        Team(name = "Argentina", continent = "América", shield = R.drawable.argentina),  
        Team(name = "Alemanha", continent = "Europa", shield = R.drawable.alemanha),  
        Team(name = "Malawi", continent = "África", shield = R.drawable.malawi),  
        Team(name = "Catar", continent = "Ásia", shield = R.drawable.catar)  
    )  
  
    val listTeams = binding.rvTeams  
    val adapter = TeamsCustomAdapter(teams)  
    listTeams.adapter = adapter  
  
    listTeams.layoutManager = LinearLayoutManager(context = this)  
}  
}
```

Executando a aplicação



Criando arrays de strings em values/strings.xml

```
➡ <string-array name="array_teams_names">
    <item>Brasil</item>
    <item>Argentina</item>
    <item>Alemanha</item>
    <item>Portugal</item>
    <item>Croácia</item>
    <item>Malawi</item>
    <item>Catar</item>
</string-array>

➡ <string-array name="array_teams_continents">
    <item>América</item>
    <item>Europa</item>
    <item>África</item>
    <item>Ásia</item>
</string-array>
```

Criando arrays de strings em values/strings.xml

```
val teams = listOf(  
    Team(name = teamsNames[0], continent = teamsContinents[0], shield = R.drawable.brasil),  
    Team(name = teamsNames[1], continent = teamsContinents[0], shield = R.drawable.argentina),  
    Team(name = teamsNames[2], continent = teamsContinents[1], shield = R.drawable.alemanha),  
    Team(name = teamsNames[5], continent = teamsContinents[2], shield = R.drawable.malawi),  
    Team(name = teamsNames[6], continent = teamsContinents[3], shield = R.drawable.catar),  
)
```

Modificando o layout

```
listTeams.layoutManager = LinearLayoutManager(context = this,  
orientation = LinearLayoutManager.INVALID_OFFSET, reverseLayout = false)
```

Modificando o layout

```
listTeams.layoutManager = LinearLayoutManager(context = this,  
orientation = LinearLayoutManager.INVALID_OFFSET, reverseLayout = false)
```

```
listTeams.layoutManager = GridLayoutManager(context = this, spanCount = 2)
```

Modificando o layout

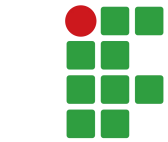
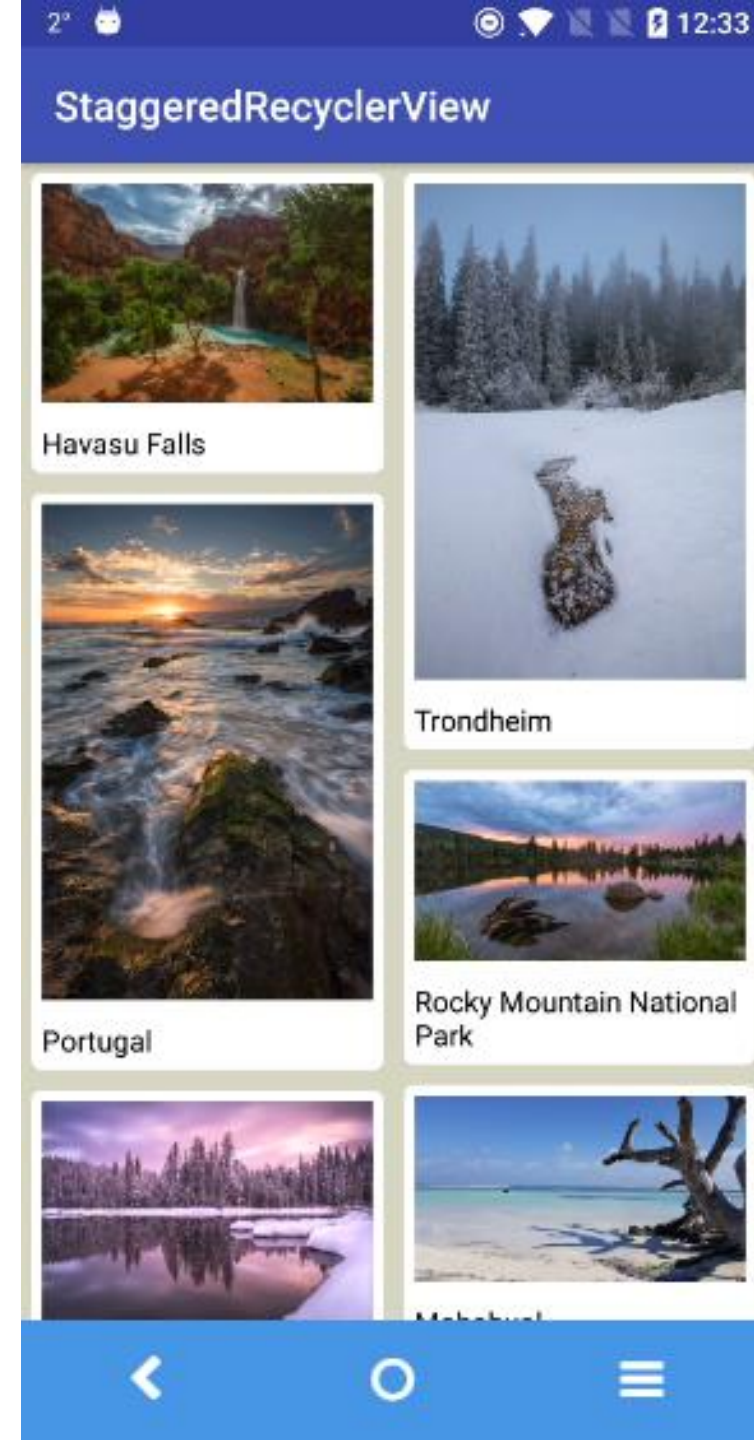
```
listTeams.layoutManager = LinearLayoutManager(context = this,  
orientation = LinearLayoutManager.INVALID_OFFSET, reverseLayout = false)
```

```
listTeams.layoutManager = GridLayoutManager(context = this, spanCount = 2)
```

```
listTeams.layoutManager = StaggeredGridLayoutManager(spanCount = 2,  
orientation = StaggeredGridLayoutManager.VERTICAL)
```

StaggeredGridLayoutManager

Documentação:
<https://developer.android.com/reference/androidx/recyclerview/widget/StaggeredGridLayoutManager>



**INSTITUTO
FEDERAL**
Pernambuco

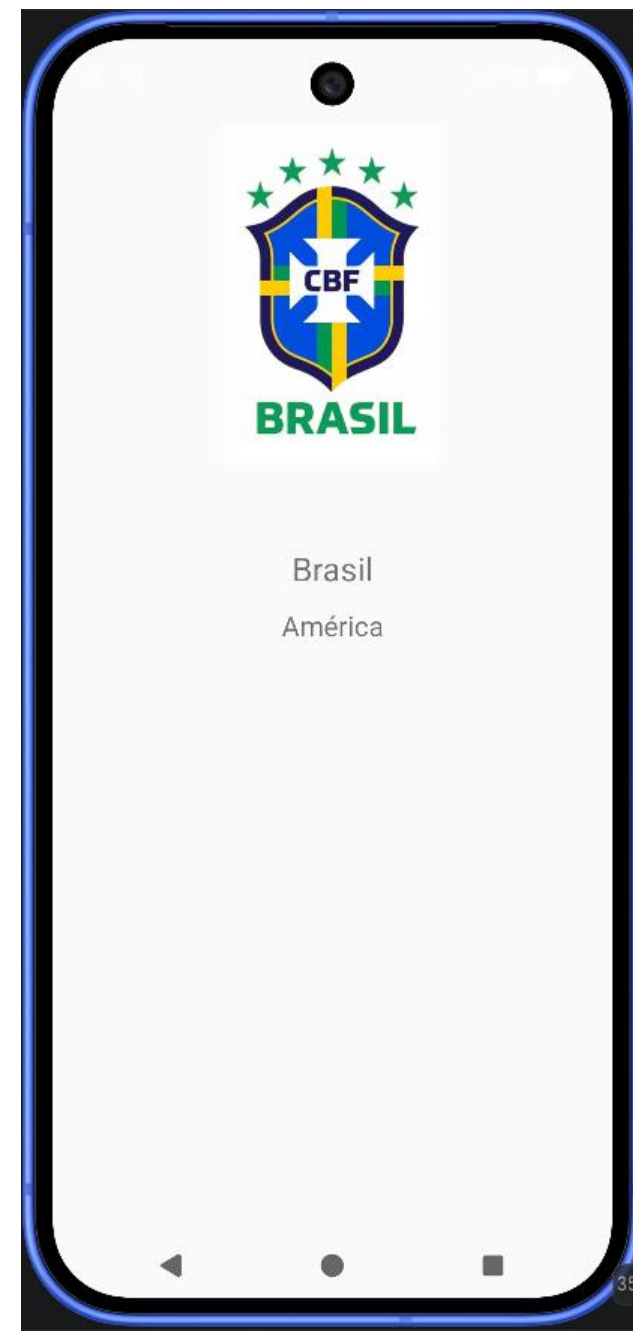
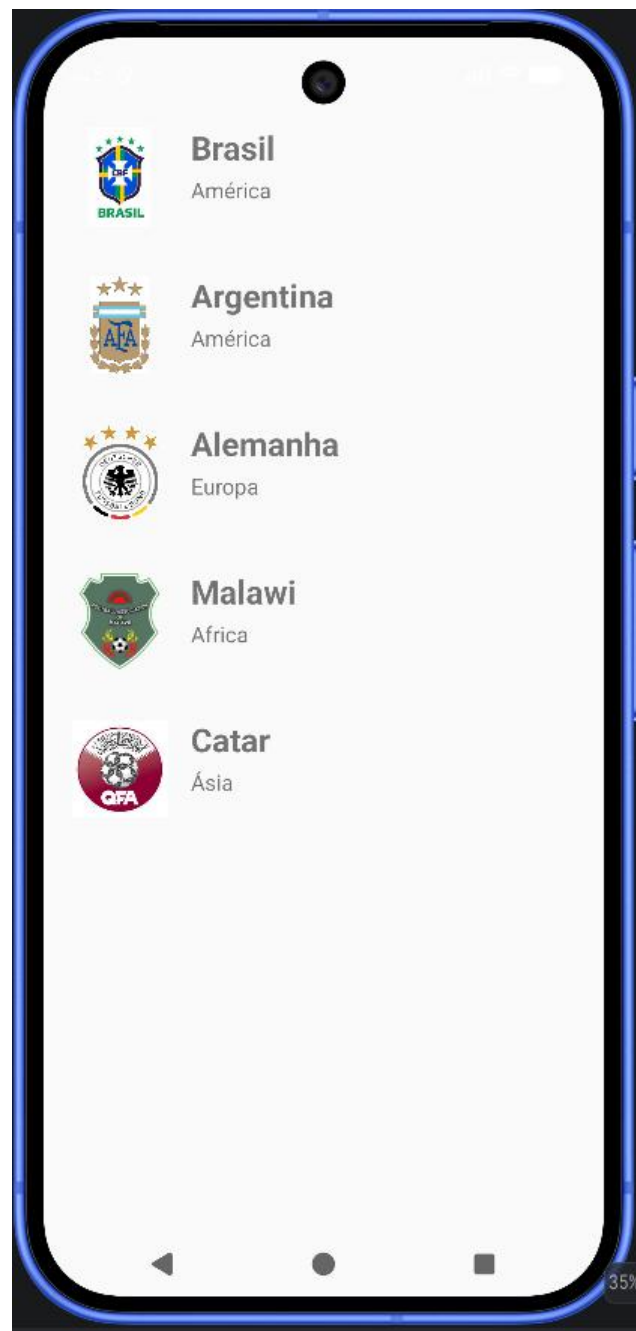
Tratando o evento de click

```
override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
    val team = teams[position]  
    holder.tvName.text = team.name  
    holder.tvContinent.text = team.continent  
    holder.ivShield.setImageResource(team.shield)  
  
    ➡ holder.itemView.setOnClickListener {  
        //inserir aqui os comandos  
    }  
}
```

Exercícios

- Crie uma segunda Activity para exibir os detalhes de um item da lista.
- Implemente o tratamento do evento de clique nos itens do RecyclerView.
- Ao clicar em um item, abra a segunda Activity.
- Envie as informações do item selecionado para a segunda Activity e apresente seus dados na tela.

Exercícios



Exercícios

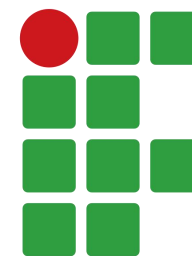
- Leia a documentação oficial sobre CardView e, em seguida, aplique o componente ao RecyclerView desenvolvido em sala, modificando o layout dos itens para que cada elemento da lista seja apresentado dentro de um CardView, explorando suas principais propriedades para melhorar a organização e a apresentação visual da lista.
 - <https://developer.android.com/develop/ui/views/layout/cardview?hl=pt-br>

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco